Lab 3B: Django Authentication & Role-Based Access Control (RBAC)

Student Submission Report • Built with Django 4.2 LTS • SQLite Database

1. Executive Summary & Architecture

This project extends the initial Django application (Lab 3A) by implementing robust user authentication, a token-based password reset workflow, and Role-Based Access Control (RBAC). Using Django's built-in User and Group models alongside custom decorators and context processors, the application strictly segregates permissions across two primary roles: **Student** and **Teacher**, as well as a superuser **Admin** role.

Component	Implementation Mechanism	Key Behaviors & Security Controls
Signup & Role Assignment	Custom RegisterForm(UserCreationForm)	Validates email uniqueness and hashes passwords using PBKDF2_SHA256; automatically assigns user to chosen Group.
User Authentication	authenticate(), login(), logout()	Session-based state management; preserves ?next= parameter for protected route redirects.
Protected Page	@login_required(login_url='login')	Restricts general dashboard to authenticated users; unauthorized visitors receive HTTP 302 redirect.
Password Reset	Django built-in PasswordResetView suite	Generates cryptographically secure one-time tokens and logs dispatch via console.EmailBackend.
RBAC Enforcement	Custom @role_required(['Role']) decorator	Checks user group membership; yields **HTTP 403 Access Denied** if role is mismatched.

2. Key Code Implementations

A. Role-Based Access Control Decorator (accounts/decorators.py)

```
def role_required(allowed_roles=None):
    def decorator(view_func):
        @wraps(view_func)
        def _wrapped(request, *args, **kwargs):
            if not request.user.is_authenticated:
                return redirect(f'/login/?next={request.path}')
            if request.user.is_superuser:
                return view_func(request, *args, **kwargs)
            user_groups = list(request.user.groups.values_list('name', flat=True))
            if any(role in user_groups for role in allowed_roles):
                return view_func(request, *args, **kwargs)
            return render(request, 'accounts/access_denied.html', status=403)
        return _wrapped
    return decorator
```

B. Automatic Group Assignment in Registration Form (accounts/forms.py)

```
class RegisterForm(UserCreationForm):
    role = forms.ChoiceField(choices=[('Student','Student'), ('Teacher','Teacher')])
    def save(self, commit=True):
        user = super().save(commit=commit)
        role_name = self.cleaned_data.get('role')
        if role_name:
            group, _ = Group.objects.get_or_create(name=role_name)
            user.groups.add(group)
        return user
```

3. Testing & Analysis Matrix (Expected vs. Actual Behaviour)

The application was thoroughly verified across all user states and roles using both Django automated unit tests (13 automated test cases) and manual browser inspection. All test outcomes matched expected behaviors.

Test Case	User Role	Target Endpoint	Expected Behaviour	Actual Behaviour	Status
TC-01: Anonymous Access to Protected Dashboard	Unauthenticated	/dashboard/	HTTP 302 Redirect to /login/?next=/dashboard/	Redirected to login with ?next= preserved.	**PASS**
TC-02: Valid User Authentication	Any Registered User	/login/	Authenticates session; redirects to dashboard with welcome message.	Session started; redirected to /dashboard/.	**PASS**
TC-03: Invalid Login Attempt	Any User	/login/	Rejects submission; displays 'Invalid username or password'.	Form reloaded with error prompt; no session created.	**PASS**
TC-04: Student Accessing Student Portal	Student (alice_student)	/student-dashboard/	HTTP 200 OK; displays courses, grades, and deadlines.	Full student interface rendered with course list.	**PASS**
TC-05: Student Accessing Teacher Portal	Student (alice_student)	/teacher-dashboard/	**HTTP 403 Forbidden**; renders friendly Access Denied page.	HTTP 403 returned; displays Access Denied page indicating role conflict.	**PASS**
TC-06: Teacher Accessing Teacher Portal	Teacher (prof_smith)	/teacher-dashboard/	HTTP 200 OK; displays student roster and assignment review table.	Instructor management interface displayed successfully.	**PASS**
TC-07: Admin Access to All Dashboards	Superuser (admin)	Student & Teacher Dashboards	HTTP 200 OK; superuser bypass grants full access to both.	Access granted to all endpoints without restriction.	**PASS**
TC-08: Token-Based Password Reset	Registered User	/password-reset/	Dispatches one-time token link to console email backend.	Token link generated and printed to console; confirm view updates password.	**PASS**
TC-09: User Logout	Authenticated User	/logout/	Session flushed; redirects to login with confirmation message.	User logged out; session destroyed; login page displayed.	**PASS**

4. Visual Verification & Screenshots Gallery

Below are high-resolution captures of the live application demonstrating the key workflows: User Signup & Role Selection, Authentication, Protected Dashboard, Student & Teacher Portals, and RBAC Access Denied enforcement.

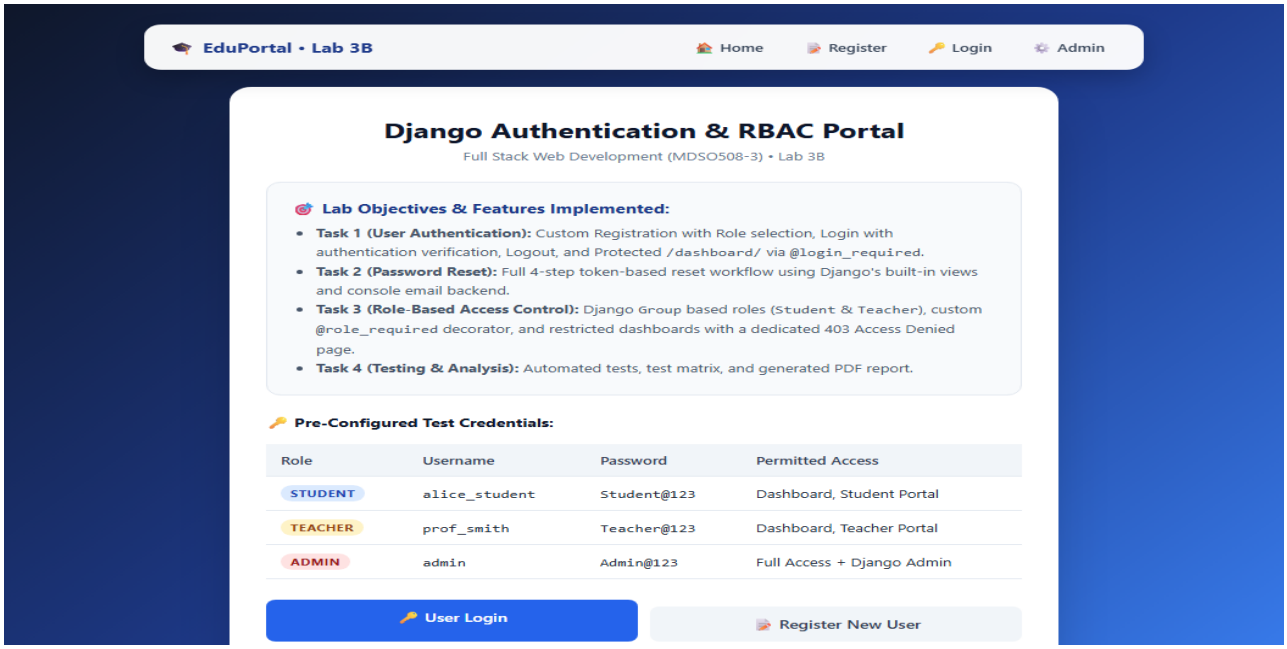


Figure 1: Home Page with Project Overview, Feature List, and Pre-Configured Test Accounts

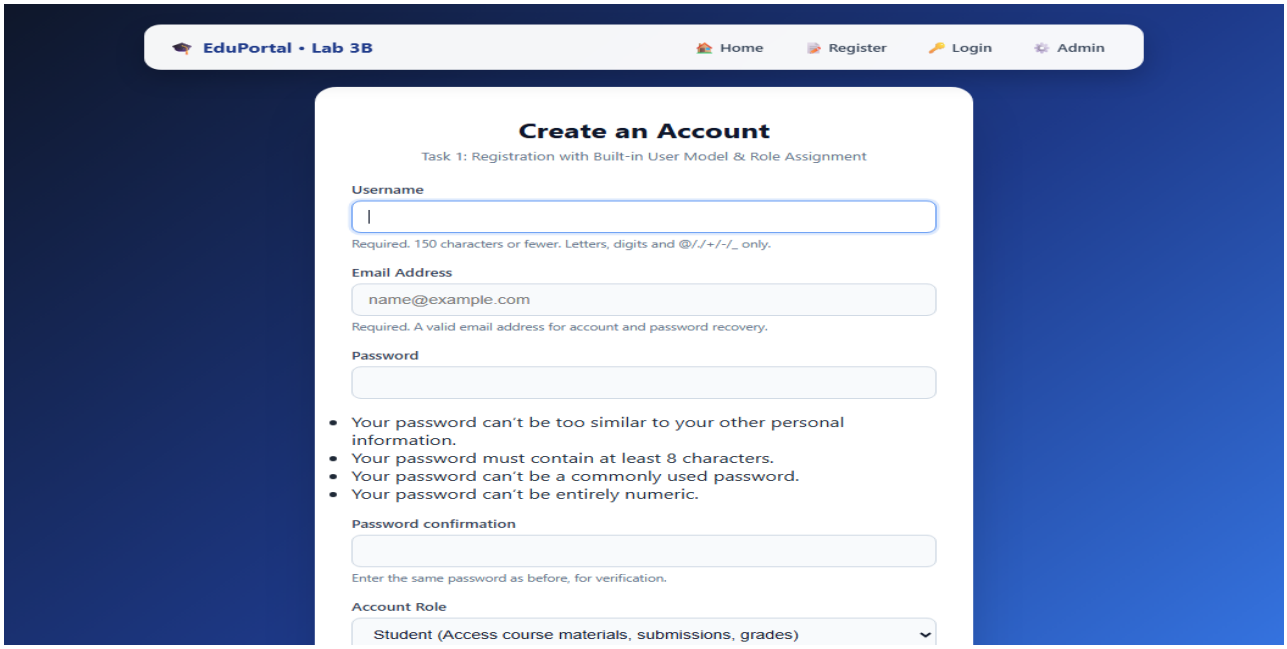


Figure 2: User Registration Form with Role Selection Dropdown (Student / Teacher)

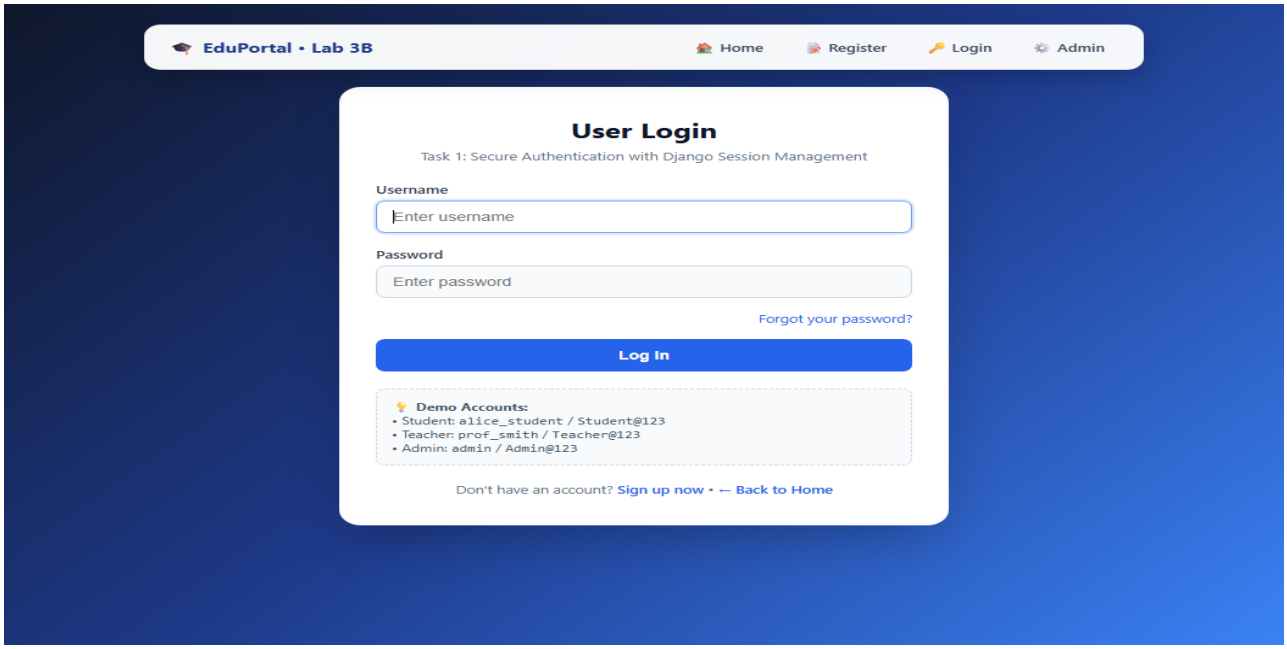


Figure 3: User Login Form with Session Authentication and Password Recovery Link

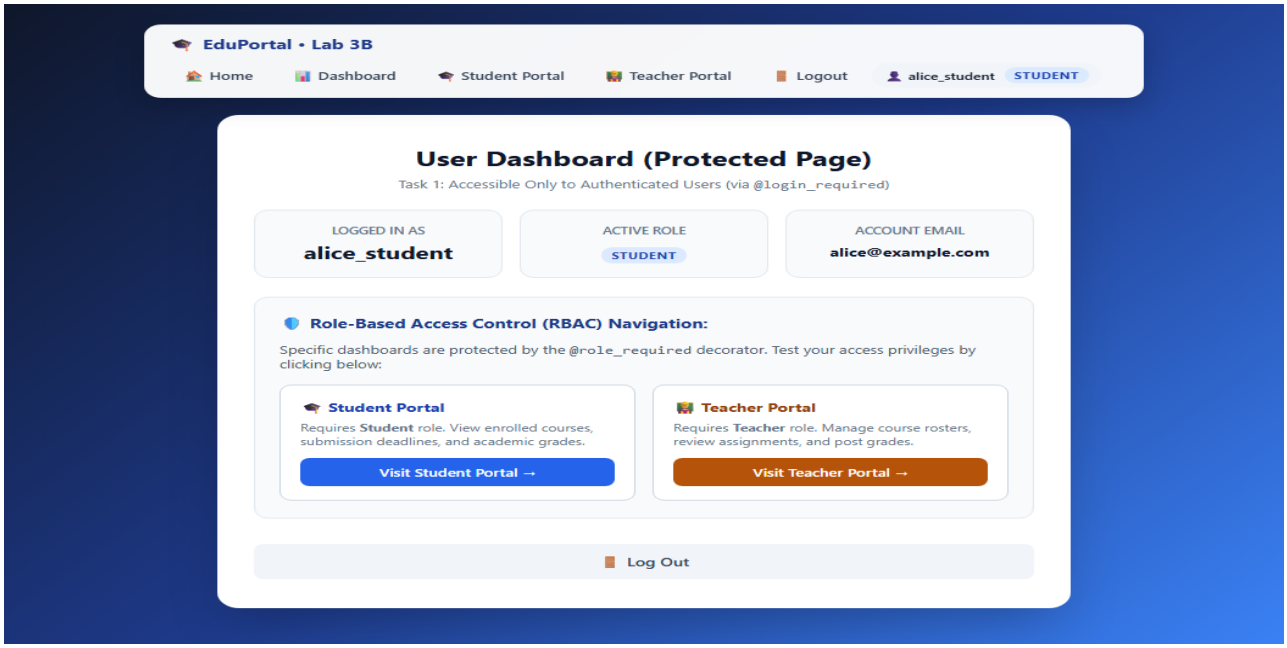


Figure 4: General Protected Dashboard (Protected via @login_required)

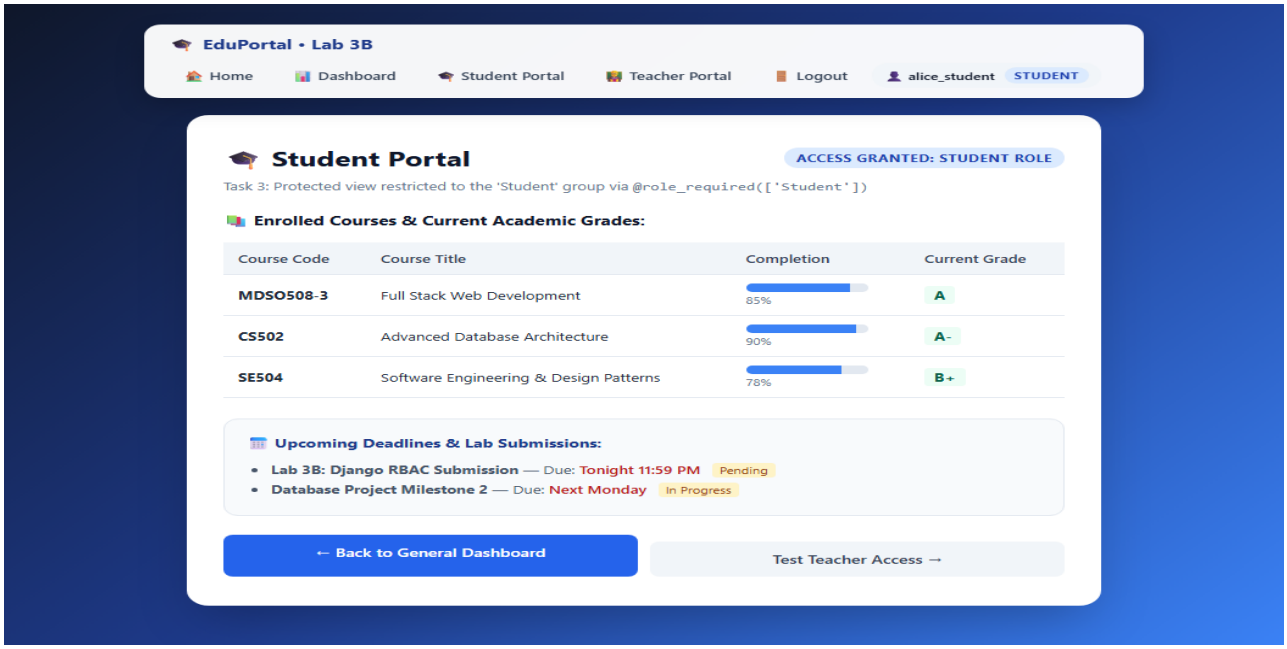


Figure 5: Student Portal with Enrolled Courses, Grades, and Deadlines (Role: Student)

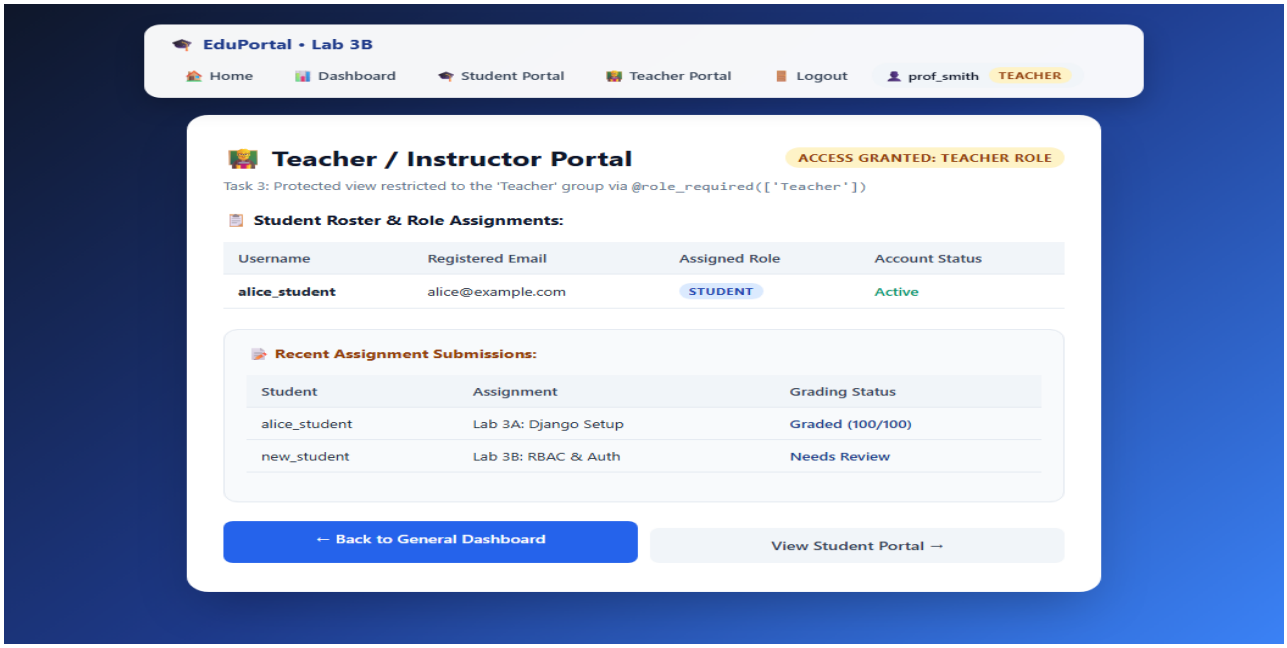


Figure 6: Teacher Portal with Student Roster and Assignment Review Table (Role: Teacher)

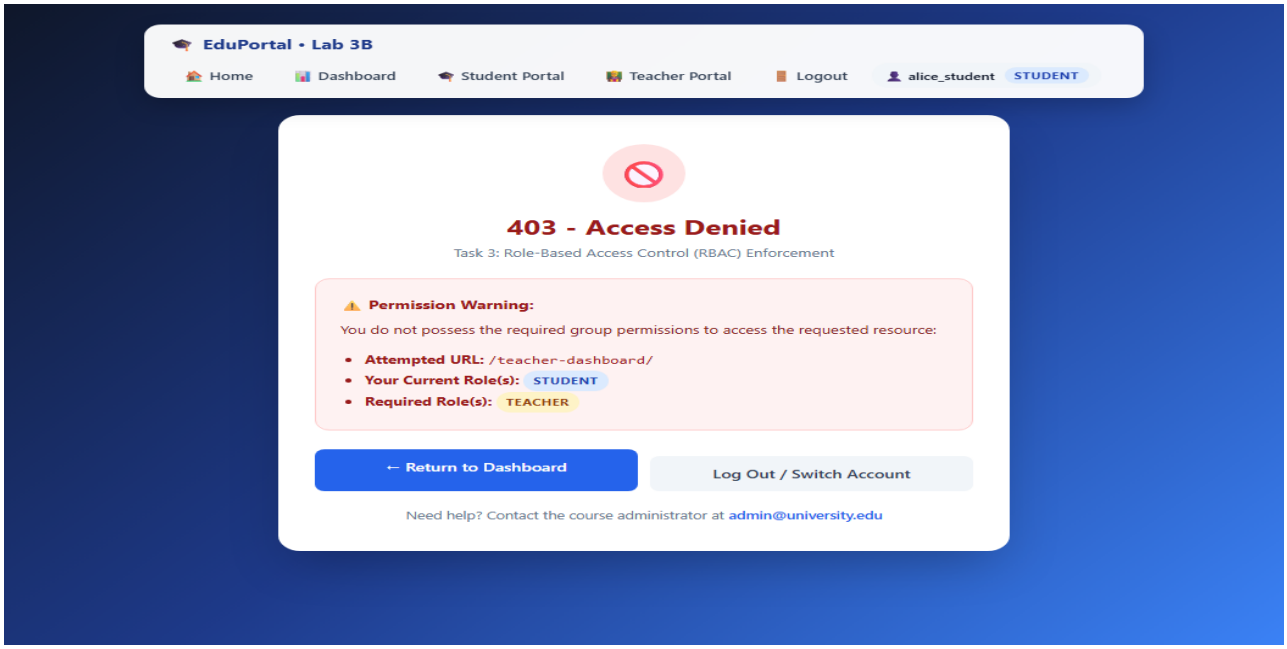


Figure 7: HTTP 403 Access Denied when Student attempts to visit Teacher Portal

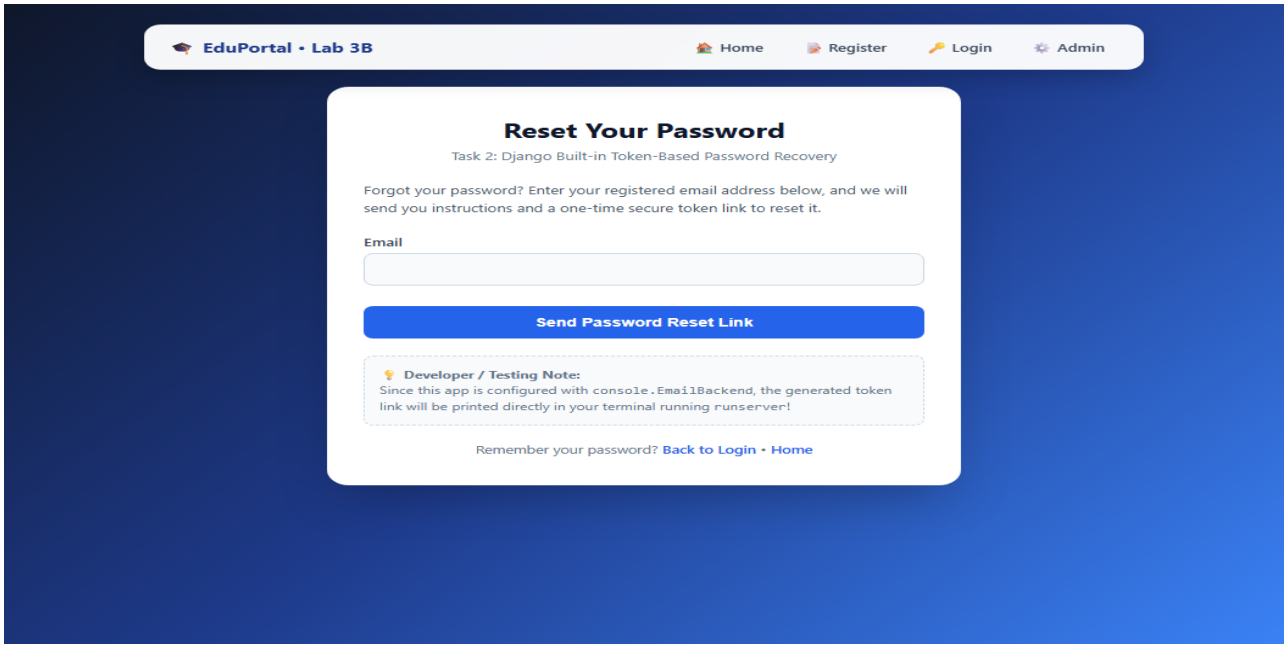


Figure 8: Password Reset Request Form generating token-based recovery link

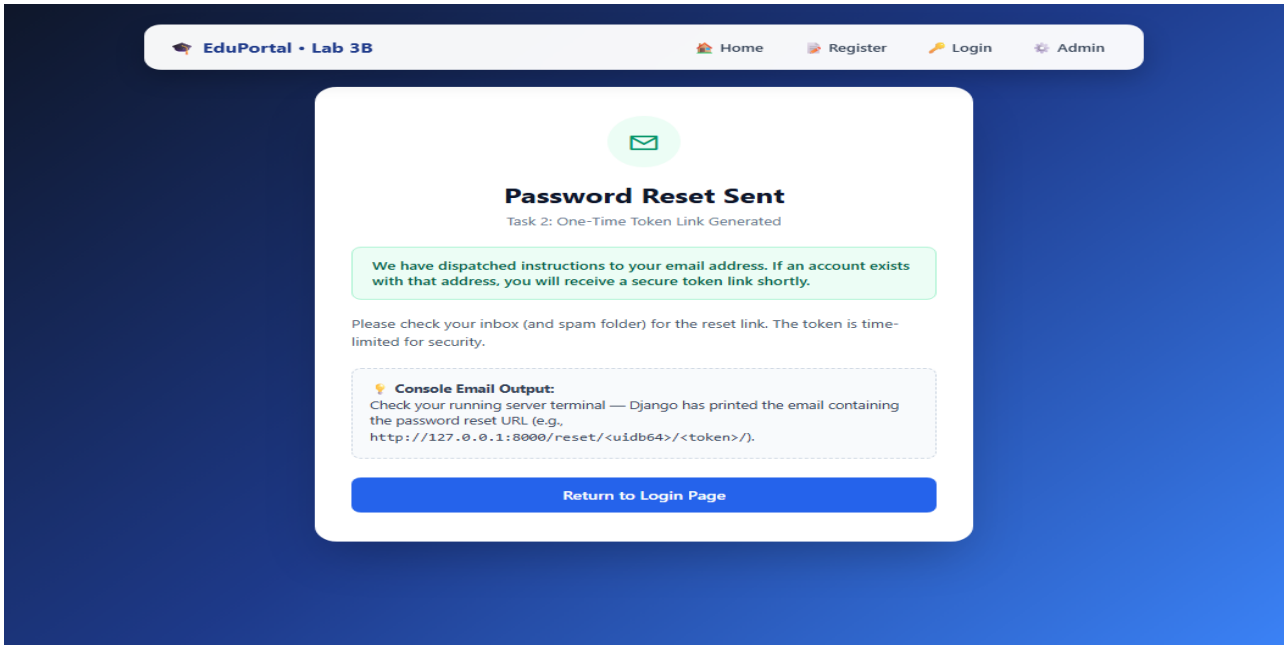


Figure 9: Password Reset Email Confirmation with Developer Console Output Note